

Reviewer Pack — Minimal Local Induction Degree Bound and Communication Compl...

Entry 8f2bf9c2f87f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-10 22:29:26 UTC

Minimal Local Induction Degree Bound and Communication Complexity Rank Variance

Entry ID: 8f2bf9c2f87f **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-10 22:29:26 UTC

1. Conjecture

Field A (mathematical branch): Arithmetic Hierarchy Theory **Field B** (complexity object): Communication Complexity - Rank Variance

Statement:

For all Boolean functions f , the minimal local induction degree bound (LIDB) of the function is linearly correlated with its communication complexity rank variance, such that $LIDB(f) = \Theta(\text{CommRankVar}(f))$.

Rationale (proposer's reasoning):

The arithmetic hierarchy provides a framework for understanding computational complexity. The local induction degree bound measures the complexity of inductive reasoning about a set of sentences. Communication complexity rank variance captures the difficulty of distributing information among parties to compute a function. This conjecture suggests that these two measures are closely related, potentially revealing a deep connection between logical reasoning and communication.

Taxonomy category: Arithmetic Hierarchy Theory × Communication Complexity - Rank Variance (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 03a78d5ea9658dce

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For all Boolean functions f , if the Pearson correlation coefficient between $LIDB(f)$ and $\text{CommRankVar}(f)$ is ≥ 0.8 for at least 24 out of 30 seeds and the mean absolute difference between $LIDB$ and CommRankVar across all seeds is ≤ 3 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - arithmetic hierarchy AND minimal local induction degree bound - communication complexity rank variance AND arithmetic hierarchy theory - minimal local induction degree bound in communication complexity AND arithmetic hierarchy

Top relevant hits considered: - [<http://arxiv.org/abs/0706.3841v1>] Arithmetic lattices and weak spectral geometry - [<http://arxiv.org/abs/1710.06598v2>] A Bounded Degree Lasserre Hierarchy with SOCP Relaxations for Global Polynomial Optimization and Applications - [<http://arxiv.org/abs/2410.15580v2>] Language Models are Symbolic Learners in Arithmetic - [<http://arxiv.org/abs/1403.8106v1>] Recent advances on the log-rank conjecture in communication complexity - [<http://arxiv.org/abs/1102.2932v2>] Applications of Monotone Rank to Complexity Theory - [<http://arxiv.org/abs/2401.14623v1>] Structure in Communication Complexity and Constant-Cost Complexity Classes - [<http://arxiv.org/abs/2111.11460v1>] On the Local Communication Complexity of Counting and Modular Arithmetic - [<http://arxiv.org/abs/2302.06539v3>] On the minimal faithful degree of Rhodes semisimple semigroups

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def lldb(f):
        n = int(math.log2(len(f)))
        if n == 0:
            return 0
        phi = [f[i] ^ f[i + 1] for i in range(len(f) - 1)]
        return max(lldb(phi), n)

    def communication_rank_variance(f):
        n = int(math.log2(len(f)))
        if n == 0:
```

```

        return 0
    phi = [f[i] ^ f[i + 1] for i in range(len(f) - 1)]
    rank_variances = [communication_rank_variance(phi), n]
    return max(rank_variances)

def pearson_correlation(lidbs, comm_rank_vars):
    if len(lidbs) != len(comm_rank_vars):
        raise ValueError("Lists must have the same length")
    n = len(lidbs)
    mean_lidb = sum(lidbs) / n
    mean_comm_rank_var = sum(comm_rank_vars) / n
    numerator = sum((lidbs[i] - mean_lidb) * (comm_rank_vars[i] - mean_comm_rank_var) for i in range(n))
    denominator = math.sqrt(sum((lidbs[i] - mean_lidb)**2 for i in range(n)) * sum((comm_rank_vars[i] - mean_comm_rank_var)**2 for i in range(n)))
    return numerator / denominator if denominator != 0 else 0

n_values = [5, 10, 15, 20, 30, 40]
lidbs = []
comm_rank_vars = []

for n in n_values:
    f = generate_boolean_function(n)
    lidb_value = lidb(f)
    comm_rank_var_value = communication_rank_variance(f)
    lidbs.append(lidb_value)
    comm_rank_vars.append(comm_rank_var_value)

correlation_coefficient = pearson_correlation(lidbs, comm_rank_vars)
mean_diff = abs(sum(lidbs) - sum(comm_rank_vars)) / len(lidbs)

return {
    "metric_name": "Pearson Correlation Coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": len(n_values),
    "n_max": max(n_values),
    "conjecture_holds": correlation_coefficient >= 0.8 and mean_diff <= 3,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif sum(1 for r in results if not r["conjecture_holds"]) / len(results) <= 0.2:

```

```

    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4a4c5967.py", line 79, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4a4c5967.py", line 55, in run_trial
    lidb_value = lidb(f)
                  ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4a4c5967.py", line 29, in lidb
    return max(lidb(phi), n)
                ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4a4c5967.py", line 29, in lidb
    return max(lidb(phi), n)
                ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4a4c5967.py", line 29, in lidb
    return max(lidb(phi), n)
                ~~~~~
[Previous line repeated 995 more times]
RecursionError: maximum recursion depth exceeded

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Re-run the test with proper error handling and ensure that it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	?	?	0	0	15139
2	propose	ollama_remote	glm4:latest	0	0	14291

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
3	preregistration	ollama_remote	glm4:latest	0	0	9928
4	novelty	ollama_remote	glm4:latest	0	0	8364
5	novelty	ollama_remote	glm4:latest	0	0	10205
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	43371
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14414
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	31289
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10714
10	judge	ollama_remote	glm4:latest	0	0	34815

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 192530 ms total latency. Provider mix: {'?': 1, 'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/8f2bf9c2f87f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/8f2bf9c2f87f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/8f2bf9c2f87f.tar.gz` (if generated)